

2.....	چکیده
3.....	فصل اول: مقدمه
3.....	۱-۱ بیان مسئله و هدف پروژه
3.....	1-2 کاربردهای پروژه
4.....	1-3 ساختار پایان نامه
5.....	فصل دوم: مفاهیم و پیشینه پژوهش
5.....	2-1 مقدمه
5.....	2-2 مبانی نظری
5.....	مدل های زبانی بزرگ (LLMs)
5.....	LangChain
5.....	FastAPI
5.....	Pydantic
6.....	VS Code Extension API
6.....	BM25
6.....	بازیابی تقویت شده ((Retriever + Retrieval-Augmented Generation
6.....	JSON Schema و اعتبارسنجی داده ها
7.....	Syntax Analysis
7.....	Code Smells و Technical Debt
7.....	Context
7.....	تعمیر خودکار برنامه ((APR
7.....	Few-Shot Prompting
8.....	Chain-of-Thought Prompting
8.....	2-3 مروری بر پژوهش های مرتبط
10.....	۴-۲ جمع بندی
11.....	فصل سوم: شرح پروژه
11.....	۱-۳ مقدمه
12.....	۲-۳ معماری کلی سیستم
14.....	۹-۳ معماری کلاینت/سرور و پروتکل ارتباطی Frontend-Backend

15.....	۱۰-۳ افزونه فرانت‌اند VS Code
16.....	۱۱-۳ API سمت سرور برای تولید پاسخ‌های مدل زبانی بزرگ
16.....	خلاصه‌ساز مسائل و بحث‌ها ((Summarizer
18.....	۳-۳ مدل داده و منابع اطلاعاتی
21.....	لایه دریافت و پردازش داده‌ها
21.....	استخراج کد منبع
21.....	گردآوری داده‌های متنی
22.....	پردازش و خلاصه‌سازی
22.....	۵-۳ راهبرد بازیابی و رتبه‌بندی
22.....	۶-۳ طراحی پرامپت و خروجی ساختاریافته
24.....	۷-۳ تولید پیشنهاد و اعمال تغییرات
26.....	۸-۳ اعتبارسنجی و کنترل کیفیت
28.....	جمع‌بندی
30.....	فصل چهارم: نتایج
30.....	۴-۱ جمع‌بندی
30.....	۴-۲ نتایج کلیدی
31.....	۴-۳ محدودیت‌ها
31.....	فصل ۵ نتیجه‌گیری و پیشنهادها
32.....	۱-۵ جمع‌بندی
32.....	۲-۵ پیشنهادهایی برای ادامه پژوهش
32.....	گسترش دامنه‌ی زبانی و فنی
32.....	بهبود دقت و کارایی
32.....	ارزیابی جامع‌تر
32.....	بررسی ابعاد انسانی و صنعتی
33.....	منابع

چکیده

با توجه به پیچیدگی روزافزون پروژه‌های نرم‌افزاری، فرآیندهای دستی مربوط به اصلاح خطا و مدیریت بدهی فنی به یکی از چالش‌های اصلی تیم‌های توسعه تبدیل شده است. این عملیات نه تنها بسیار زمان‌بر است، بلکه نیازمند درک عمیق از کل پایگاه کد بوده و همواره مستعد خطای انسانی است. در نتیجه، نیاز به یک ابزار هوشمند و خودکار که بتواند به توسعه‌دهندگان در شناسایی، تحلیل و ارائه راه‌حل برای بهبود کیفیت کد کمک کند، به شدت احساس می‌شود.

در این پروژه، یک سامانه جامع برای تحلیل و اصلاح خودکار کد با معماری سرویس‌دهنده-سرویس‌گیرنده ارائه شده است. این سامانه شامل یک افزونه برای محیط توسعه Visual Studio Code به عنوان واسط کاربری و یک API بک‌اند برای اجرای پردازش‌ها است. رویکرد پیشنهادی با یک پایپ‌لاین هوشمند، با بهره‌گیری از تکنیک‌های بازیابی اطلاعات برای یافتن زمینه مرتبط و استفاده از مدل‌های زبانی بزرگ (LLMs)، پیشنهادهای اصلاحی دقیق و ساختاریافته تولید می‌کند. خروجی مدل پیش از ارائه به کاربر، طی یک فرآیند اعتبارسنجی خودکار از نظر ساختاری و نحوی بررسی می‌شود تا از کیفیت آن اطمینان حاصل گردد.

نتیجه این پژوهش، یک چارچوب عملی و قابل توسعه است که فرآیند اصلاح کد را به صورت هوشمند خودکارسازی می‌کند. این سامانه با یکپارچه‌سازی در محیط کاری توسعه‌دهنده، به طور قابل توجهی زمان و تلاش دستی مورد نیاز برای شناسایی و رفع مشکلات کیفی کد را کاهش می‌دهد. استفاده از مکانیزم بازیابی اطلاعات، دقت و آگاهی از زمینه را در پیشنهادهای تولیدشده توسط مدل زبانی افزایش داده و منجر به ارائه راه‌حل‌های کاربردی‌تر می‌شود. در نهایت، این ابزار نه تنها به عنوان یک دستیار هوشمند برای اصلاح لحظه‌ای کد عمل می‌کند، بلکه با شناسایی پیشگیرانه بدهی فنی، به بهبود مستمر کیفیت نرم‌افزار در طول چرخه حیات آن کمک می‌نماید.

واژگان کلیدی: مدل‌های زبانی بزرگ، تعمیر خودکار برنامه، بدهی فنی، نقاط ضعف کد، بازیابی اطلاعات، بازیابی تقویت‌شده

فصل اول: مقدمه

۱-۱ بیان مسئله و هدف پروژه

با افزایش مقیاس و پیچیدگی سامانه‌های نرم‌افزاری، اتکا به بازبینی‌های دستی کد موجب اتلاف زمان، ناهمگنی در کیفیت و تشدید بدهی فنی می‌شود. این فرایند، علاوه بر نیاز به درک عمیق از پایگاه کد، مستعد خطای انسانی است و پیگیری همگام استانداردها و بهترین رویه‌ها را دشوار می‌سازد. از این رو نیاز به ابزاری خودکار و هوشمند برای شناسایی، تحلیل و ارائهٔ پیشنهادهای اصلاحی احساس می‌شود تا کیفیت کد ارتقا یافته و بار شناختی توسعه‌دهندگان کاهش یابد.

هدف اصلی این پژوهش، طراحی و پیاده‌سازی یک سامانه برای پیشنهاد **اصلاحات کد** است که با بهره‌گیری از **مدل‌های زبانی بزرگ (LLMs)** و تکنیک‌های **بازیابی اطلاعات** بتواند در محیط توسعه‌دهندگان (به‌ویژه VS Code) یک دستیار هوشمند برای بهبود کیفیت کد فراهم سازد.

۱-۲ کاربردهای پروژه

این پژوهش علاوه بر کمک به افزایش کیفیت کد در پروژه‌های نرم‌افزاری، دارای کاربردهای متعددی است، از جمله:

- کاهش هزینه و زمان توسعه از طریق خودکارسازی فرآیند بازبینی و اصلاح کد.
- پیشگیری از ایجاد بدهی فنی با شناسایی زود هنگام مشکلات ساختاری و کدهای مستعد خطا.
- بهبود تجربه برنامه‌نویسان از طریق یکپارچه‌سازی دستیار هوشمند در محیط توسعه (VS Code).
- افزایش پایداری و مقیاس‌پذیری نرم‌افزارها به دلیل اصلاح مستمر و هدفمند کدها.
- کاربرد در صنعت برای تیم‌های بزرگ نرم‌افزاری که نیازمند کاهش بار دستی کدنویسان در بازبینی هستند.

1-3 ساختار پایان نامه

این پایان نامه در پنج فصل تنظیم شده است:

- **فصل اول (مقدمه):** به بیان ضرورت پژوهش، اهداف و کاربردهای پروژه پرداخته می شود.
- **فصل دوم (مفاهیم):** شامل مرور ادبیات پژوهش و مبانی نظری مرتبط با مدل های زبانی بزرگ، روش های بازیابی اطلاعات، خلاصه سازی و روش های اعتبارسنجی خروجی است.
- **فصل سوم (شرح پروژه):** معماری سامانه، اجزای اصلی و مراحل پیاده سازی شامل برچسب گذاری خودکار، بازیابی و رتبه بندی، طراحی پرامپت، تولید پیشنهاد و کنترل کیفیت معرفی می شوند.
- **فصل چهارم (نتایج):** در این فصل، نتایج حاصل از ارزیابی سیستم شامل کارایی در شناسایی مشکلات، دقت پیشنهادهای اصلاحی و تأثیر آن بر کاهش زمان توسعه ارائه می شود. همچنین یافته ها تحلیل شده و محدودیت های روش پیشنهادی مورد بحث قرار می گیرد.
- **فصل پنجم (نتیجه گیری و پیشنهادات):** این فصل به جمع بندی کلی پژوهش، پاسخ به سوالات تحقیق و ارائه نتیجه گیری نهایی می پردازد. در ادامه، پیشنهادهایی برای گسترش و بهبود سیستم در کارهای آتی ارائه خواهد شد.

فصل دوم: مفاهیم و پیشینه پژوهش

2-1 مقدمه

در این فصل به معرفی ابزارها، کتابخانه‌ها، چارچوب‌ها و الگوریتم‌هایی پرداخته می‌شود که در طراحی و پیاده‌سازی سامانه پیشنهادی مورد استفاده قرار گرفته‌اند. این مباحث علاوه بر تعریف مفاهیم پایه، نقش هرکدام را در حل مسئله اصلاح خودکار کد روشن می‌سازند.

2-2 مبانی نظری

مدل‌های زبانی بزرگ (LLMs)

مدل‌های زبانی بزرگ (Large Language Models) شبکه‌های عصبی عمیقی هستند که بر حجم عظیمی از داده‌های متنی آموزش دیده‌اند و توانایی درک و تولید زبان طبیعی را دارند. این مدل‌ها می‌توانند متون طولانی را تحلیل کرده و خروجی‌هایی روان و معنادار تولید کنند. در این پروژه، مدل‌های زبانی برای پیشنهاد تغییرات در کد و بازنویسی بخش‌های معیوب مورد استفاده قرار گرفته‌اند.

LangChain

LangChain یک چارچوب متن‌باز است که امکان مدیریت ورودی‌ها و خروجی‌های مدل‌های زبانی را به صورت ساختاریافته فراهم می‌کند. این ابزار کمک می‌کند پرامپت‌ها به شکل منظم طراحی شده و نتایج در قالب JSON بازگردانده شوند. در پروژه حاضر، LangChain برای ایجاد **bounded-context prompt** و دریافت خروجی ساختاریافته استفاده شده است.

FastAPI

FastAPI یک فریم‌ورک مدرن و سریع برای توسعه سرویس‌های وب مبتنی بر REST است که با زبان پایتون پیاده‌سازی می‌شود. این ابزار به دلیل سرعت بالا، سادگی در تعریف مسیرها و پشتیبانی از مستندسازی خودکار، برای طراحی بخش سرور پروژه به کار گرفته شده است. در سامانه حاضر، FastAPI وظیفه پردازش درخواست‌ها، اجرای منطق پیچیده و ارتباط با مدل زبانی را بر عهده دارد.

Pydantic

Pydantic کتابخانه‌ای در پایتون است که برای اعتبارسنجی داده‌ها و تضمین صحت ساختار آن‌ها طراحی شده است. این ابزار به ویژه در کار با داده‌های JSON کاربرد دارد و مانع از ایجاد خطاهای ناشی از داده‌های ناقص یا ناسازگار می‌شود. در این پروژه، Pydantic برای بررسی خروجی مدل زبانی و اطمینان از مطابقت آن با قالب تعریف‌شده استفاده شده است.

VS Code Extension API

Extension API مجموعه‌ای از ابزارها و توابع رسمی است که امکان توسعه افزونه برای محیط Visual Studio Code را فراهم می‌کند. با استفاده از این API می‌توان قابلیت‌های جدیدی به ویرایشگر اضافه کرد و نتایج حاصل از پردازش‌های سمت سرور را مستقیماً در محیط توسعه نمایش داد. در این پروژه، افزونه طراحی‌شده نقش رابط تعاملی میان کاربر و سامانه را ایفا می‌کند.

BM25

BM25 یک الگوریتم بازیابی اطلاعات مبتنی بر کلمات کلیدی است که برای رتبه‌بندی اسناد بر اساس شباهت به یک پرسش استفاده می‌شود. این الگوریتم بر پایه فراوانی واژه‌ها و طول سند عمل می‌کند و یکی از پرکاربردترین روش‌ها در موتورهای جست‌وجو محسوب می‌شود. در پروژه حاضر، BM25 برای یافتن بخش‌های مرتبط از کد و مسائل نرم‌افزاری به کار گرفته شده است.

بازیابی تقویت‌شده (Retriever + Retrieval-Augmented Generation)

یکی از چالش‌های LLM‌ها محدودیت حافظه زمینه است. برای رفع این مشکل، از معماری‌هایی موسوم به بازیابی تقویت‌شده استفاده می‌شود. در این رویکرد، بخش «Retriever» وظیفه دارد قطعات مرتبط از مخزن یا اسناد را با استفاده از جست‌وجو بازیابی کند و بخش «Generator» با استفاده از آن قطعات به همراه پرامپت، پاسخ نهایی یا پیچ کد را تولید می‌کند. این روش کمک می‌کند مدل از دانش بیرونی و سوابق پروژه بهره‌بردار شود.

JSON Schema و اعتبارسنجی داده‌ها

JSON Schema استاندارد برای تعریف ساختار داده‌های JSON است. این استاندارد مشخص می‌کند که داده چه فیلدهایی باید داشته باشد، نوع داده هر فیلد چیست و چه قیودی باید رعایت شود. در زبان پایتون، کتابخانه Pydantic ابزار رایجی برای پیاده‌سازی این اعتبارسنجی است. استفاده از این

سازوکار باعث می‌شود خروجی مدل‌های زبانی ساختاریافته و معتبر باشد و خطاهای ناشی از داده‌های ناقص کاهش یابد.

Syntax Analysis

تحلیل نحوی (Syntax Analysis) فرآیندی در علوم کامپیوتر است که ساختار کد را از نظر مطابقت با قواعد زبان برنامه‌نویسی بررسی می‌کند. در پروژه حاضر، پس از اعمال تغییرات پیشنهادی مدل زبانی، تحلیل نحوی بر روی کد انجام می‌شود تا از صحت ساختار و جلوگیری از بروز خطاهای نحوی اطمینان حاصل گردد.

Code Smells و Technical Debt

بدهی فنی (Technical Debt) اصطلاحی است که به مشکلات و ضعف‌های موجود در کد اطلاق می‌شود که ممکن است در کوتاه‌مدت مانع عملکرد برنامه نشوند، اما در بلندمدت هزینه نگهداری و توسعه را افزایش می‌دهند.

از سوی دیگر، **نقاط ضعف کد (Code Smell)** به نشانه‌هایی گفته می‌شود که بیانگر کیفیت پایین طراحی یا ضعف ساختاری در کد هستند، مانند توابع طولانی یا تکرار زیاد کد.

Context

در مدل‌های زبانی، context به مجموعه اطلاعاتی گفته می‌شود که در ورودی مدل قرار می‌گیرد و مبنای تولید پاسخ است. در حوزه تعمیر برنامه، context می‌تواند شامل کد منبع، پیغام خطا، تست‌ها یا توضیحات مربوط به issue باشد و نقش مهمی در عملکرد صحیح مدل ایفا می‌کند.

تعمیر خودکار برنامه (APR)

تعمیر خودکار برنامه (APR) یا Automated Program Repair به فرآیندی اطلاق می‌شود که در آن، سیستم به صورت خودکار باگ را شناسایی، اصلاح پیشنهادی را تولید، و صحت آن را اعتبارسنجی می‌کند. هدف APR کاهش هزینه و زمان نگهداری نرم‌افزار بدون دخالت مستقیم انسان است.

Few-Shot Prompting

در few-shot prompting، مدل زبانی با چند مثال حل شده مشابه درون پرامپت هدایت می شود تا بتواند مسئله جدید را بر اساس الگوهای دیده شده حل کند. این روش بدون نیاز به آموزش مجدد مدل، رفتار آن را با نمونه ها تنظیم می کند.

Chain-of-Thought Prompting

chain-of-thought prompting مدلی از پرامپت گذاری است که در آن مدل تشویق می شود مسیر استدلال خود را مرحله به مرحله بیان کند. این روش برای بهبود دقت مدل در وظایف پیچیده مانند تحلیل باگ و تولید پچ کاربرد دارد.

2-3 مروری بر پژوهش های مرتبط

Xu و همکاران در تحقیق ThinkRepair **نیاز به منبع** رویکردی Self-Directed را پیشنهاد کردند که با بهره گیری از Chain-of-Thought و few-shot prompting توانایی استدلالی مدل های زبانی را در تولید پچ افزایش می دهد. نتایج نشان داد که افزودن استدلال گام به گام، کیفیت اصلاحات را ارتقا می دهد. هرچند، این روش به داده های برچسب خورده برای مثال های آموزشی وابسته است و برای کار در مقیاس بزرگ محدودیت دارد.

Wang و همکاران نشان دادند **نیاز به منبع** که داده های حاصل از Pull Requests و Issues در GitHub می توانند برای هدایت فرآیند رفع باگ ها به کار گرفته شوند. آن ها دریافتند که افزودن این اطلاعات متنی به context ورودی مدل، باعث می شود LLM ها درک بهتری از منطق خطا و نیازمندی های اصلاح پیدا کنند. با وجود این، رویکرد آن ها فاقد یک سازوکار استاندارد برای خلاصه سازی و انتخاب گزینشی داده ها بود، که موجب افزایش نویز در ورودی می شود.

Chen و همکاران در مطالعه RepoBugs **نیاز به منبع** به بررسی عملکرد LLM ها در رفع باگ های سطح مخزن پرداختند. یافته ها نشان دادند که عملکرد مدل ها در صورتی که تنها context محلی (در سطح تابع) دریافت کنند، ضعیف است. برای رفع این مشکل، آن ها روش RLCE (Repository-Level Context Extraction) را معرفی کردند که با انتخاب بخش های مرتبط از مخزن، موفقیت تعمیر را تا 160٪ افزایش داد. هرچند، این روش نیازمند الگوریتم های پیچیده ی استخراج و محاسبات پرهزینه است.

Li و همکاران در تحقیق GIANTREPAIR **نیاز به منبع** ترکیب تحلیل ایستا و مدل‌های زبانی را برای تولید پچ پیشنهاد کردند. در این روش ابتدا یک Patch Skeleton از طریق تحلیل ایستا ساخته می‌شود و سپس LLM آن را تکمیل می‌کند. این ترکیب توانست دقت بیشتری نسبت به LLM تنها ارائه دهد. محدودیت اصلی این رویکرد، پیچیدگی پیاده‌سازی و نیاز به ابزارهای اضافی تحلیل برنامه است.

Jin و همکاران ابزار InferFix **نیاز به منبع** را معرفی کردند که از **بازیابی قطعات کد مرتبط** و ترکیب آن‌ها با پرامپت‌های مدل زبانی بهره می‌گیرد. این سیستم یک رویکرد انتها-به-انتها برای APR فراهم کرد و نشان داد که بازیابی داده‌های مرتبط می‌تواند دقت پچ‌ها را افزایش دهد. با این حال، این روش در انتخاب داده‌های غیرمرتبط مشکل داشت که گاهی موجب تولید پچ‌های نادرست می‌شد.

Mu و همکاران در EXPERPAIR **نیاز به منبع** از رویکرد حافظه دوگانه برای ذخیره و استفاده از تجربیات گذشته در تعمیر باگ‌ها استفاده کردند. نتایج نشان داد که بهره‌گیری از حافظه تاریخی می‌تواند کیفیت تعمیر در مخازن بزرگ را افزایش دهد. با وجود این، این روش به شدت به حجم و کیفیت داده‌های قبلی وابسته است و در پروژه‌های کوچک یا تازه‌تأسیس کاربرد محدودی دارد.

Xie و همکاران در SWE-Fixer **نیاز به منبع** یک چارچوب متن‌باز ساخته‌اند که هدف آن رفع خودکار مسائل گزارش شده در مخازن GitHub با استفاده از مدل‌های زبانی است. ایده‌ی اصلی این است که به جای وابستگی به مدل‌های اختصاصی و بسته، از مدل‌های متن‌باز استفاده شود تا قابلیت تکرارپذیری، شفافیت و دسترسی بهتری داشته باشیم. SWE-Fixer یک خط لوله (pipeline) دو مرحله‌ای دارد: اول بازیابی فایل‌های کد (code file retrieval) با استفاده از یک استراتژی coarse-to-fine که ابتدا با BM25 کاندیدهایی را انتخاب می‌کند و سپس مدل بازیابی سبک‌تر فاین‌تیون شده را برای تشخیص فایل‌های دقیق‌تر به کار می‌برد؛ دوم ویرایش کد (code editing) که پس از تعیین فایل‌ها، مدل ویرایشگر کد را برای تولید patch به کار می‌گیرد. از مزایای مهم SWE-Fixer این است که با تعداد دو فراخوانی مدل (یک بار برای بازیابی فایل و یک بار برای ویرایش) به‌ازای هر نمونه، نسبت به خیلی از روش‌های قبلی مقرون‌به‌صرفه‌تر است.

Blaauwendraad و همکاران نشان دادند **نیاز به منبع** که LLM‌ها توانایی شناسایی Code Smells مانند متدهای طولانی یا کلاس‌های بیش از حد بزرگ را دارند. Wadhwa و همکاران نیز ابزار **نیاز CORE** **به منبع** را ارائه کردند که با استفاده از LLM مشکلات کیفیتی شناسایی‌شده توسط تحلیل ایستا (مانند SonarQube) را رفع می‌کند. هرچند، هر دو پژوهش فاقد یک سازوکار جامع برای ترکیب تشخیص و اصلاح بودند.

Sheikhaei و همکاران به شناسایی بدهی فنی خوداظهار (SATD) با استفاده از LLM ها پرداختند **نیاز** به منبع و محدودیت‌هایی مانند نبود معیارهای ارزیابی دقیق را مطرح کردند. Shivashankar. این مطالعه نشان‌دهنده‌ی پتانسیل بالای LLM ها در مدیریت بدهی فنی است، اما همچنان به داده‌های متنی باکیفیت و تنظیم دقیق نیاز دارد.

بررسی مطالعات پیشین نشان می‌دهد که هرچند دستاوردهای قابل توجهی در زمینه تعمیر خودکار برنامه و بهبود کیفیت کد با استفاده از LLM ها حاصل شده است، اما شکاف‌های مهمی همچنان باقی مانده است:

1. **مدیریت کارآمد context:** اغلب پژوهش‌ها بر انتخاب یا گسترش context تأکید دارند، اما سازوکارهای مؤثر برای خلاصه‌سازی و ترکیب چند منبع اطلاعاتی (issues, PRs, discussions, code) ارائه نشده است.
2. **اعتبارسنجی خودکار خروجی‌ها:** بسیاری از رویکردها فاقد مکانیزم‌های سیستماتیک برای اعتبارسنجی خروجی مدل‌ها هستند (مانند تحلیل نحوی یا اجرای تست).
3. **کاربردپذیری در محیط توسعه واقعی:** گرچه SWE-Fixer و SWE-Bench قدم‌هایی در این جهت برداشته‌اند، اما هنوز ابزارهایی که به‌صورت عملی در محیط IDE یکپارچه شوند کم‌یاب هستند.

پژوهش حاضر با طراحی سیستمی مبتنی بر LLM که شامل مدیریت context، خلاصه‌سازی داده‌های متنی، اعتبارسنجی نحوی و تست خودکار، و یک افزونه‌ی VS Code برای تعامل مستقیم با توسعه‌دهنده است، به پر کردن این شکاف‌ها کمک می‌کند.

۲-۴- جمع‌بندی

در این فصل ابزارها، چارچوب‌ها و الگوریتم‌های کلیدی مورد استفاده در پروژه معرفی شدند. این مفاهیم شامل مدل‌های زبانی بزرگ، ابزارهای مدیریت داده و پرامپت، روش‌های بازیابی اطلاعات، مکانیزم‌های خلاصه‌سازی و اعتبارسنجی و همچنین مفاهیم بدهی فنی و نقاط ضعف کد بودند. همچنین پژوهش‌های مرتبط بررسی شدند.

فصل سوم: شرح پروژه

۳-۱- مقدمه

پس از بررسی مبانی نظری و مرور مفاهیم در فصل دوم، این فصل به تشریح دقیق طراحی فنی، معماری و جزئیات پیاده‌سازی سیستم پیشنهادی برای اصلاح خودکار کد می‌پردازد. هدف اصلی این فصل، ارائه یک نقشه راه کامل از عملکرد سیستم است که نشان می‌دهد چگونه اجزای مختلف برای دستیابی به یک راه‌حل یکپارچه با یکدیگر همکاری می‌کنند.

معماری این سیستم بر پایه یک مدل سرویس‌دهنده-سرویس‌گیرنده (Client-Server) طراحی شده است تا تعاملات کاربر از فرآیندهای محاسباتی سنگین مجزا باشد. در سمت سرویس‌گیرنده، یک افزونه برای محیط توسعه یکپارچه Visual Studio Code پیاده‌سازی شده است تا ابزار به صورت مستقیم و بدون ایجاد اختلال در جریان کاری برنامه‌نویس، در دسترس او قرار گیرد. در سمت سرویس‌دهنده، یک API بک‌اند (Backend) وظیفه مدیریت و اجرای تمام منطق‌های پیچیده، از تحلیل اولیه کد تا تولید نهایی پیشنهادهای اصلاحی مبتنی بر مدل زبانی بزرگ (LLM)، را بر عهده دارد.

یکی از ویژگی‌های کلیدی این پروژه، رویکرد پیشگیرانه آن در شناسایی نقاط قابل بهبود است. سیستم در فاز اولیه، به صورت خودکار مخزن کد و مشکلات (Issues) مرتبط با آن را تحلیل کرده و فایل‌ها را بر اساس انواع بدهی فنی (Technical Debt) و نقاط ضعف کد (Code Smells) برچسب‌گذاری می‌کند. این فرآیند، دو حالت عملیاتی اصلی را امکان‌پذیر می‌سازد: یک اجرای خودکار اولیه که بر روی موارد از پیش شناسایی‌شده با استفاده از برچسب‌گذاری متمرکز است و یک حالت دستی که در آن توسعه‌دهنده می‌تواند تحلیل را برای هر فایل یا مشکل (Issue) دلخواه به طور مستقیم از طریق افزونه فعال کند.

در ادامه این فصل، ابتدا معماری کلی سیستم با تشریح نقش افزونه VS Code و API بک‌اند معرفی می‌شود. سپس، متدولوژی مورد استفاده برای برچسب‌گذاری خودکار بدهی فنی شرح داده خواهد شد. پس از آن، گردش کار سیستم و پایپ‌لاین چندمرحله‌ای بک‌اند به صورت گام‌به‌گام، شامل فازهای گردآوری داده، بازیابی اطلاعات مرتبط، تولید کد و اعتبارسنجی، به تفصیل مورد بررسی قرار می‌گیرد.

ابزارهای پیاده‌سازی:

1. **FastAPI** نیاز به منبع
یک فریم‌ورک وب مدرن در پایتون که برای توسعه سرویس‌های **RESTful API** استفاده شده است.
2. **Pydantic** نیاز به منبع
کتابخانه‌ای برای اعتبارسنجی داده‌ها و بررسی خروجی‌های JSON در برابر شمای تعریف‌شده.
3. **LangChain** نیاز به منبع
چارچوبی برای ساخت پرامپت‌ها (prompts) و هماهنگ‌سازی ورودی/خروجی مدل‌های زبانی. این ابزار امکان تعریف **Bounded-Context Prompt** و دریافت خروجی ساختاریافته را فراهم می‌کند. همین‌طور الگوریتم BM25 نیز توسط این کتابخانه قابل اجرا است.
4. **OpenAI / Gemini API Clients**
کتابخانه‌های رسمی یا واسطه‌های HTTP برای فراخوانی مدل‌های زبانی بزرگ (LLM). از این طریق پرامپت آماده‌شده ارسال شده و خروجی JSON بازگردانده می‌شود.
5. **Visual Studio Code Extension API** نیاز به منبع
بستر رسمی برای توسعه افزونه‌ها در VS Code؛ از طریق این API می‌توان امکاناتی مثل دسترسی به فایل‌ها، نمایش نتایج در محیط ویرایشگر و ارتباط با بک‌اند را پیاده‌سازی کرد.
- 6.

۳-۲- معماری کلی سیستم

معماری سامانه‌ی پیشنهادی بر مبنای یک الگوی چندلایه طراحی شده است تا جداسازی وظایف تضمین گردد. این معماری با جداسازی واضح وظایف، امکان می‌دهد تا هر مؤلفه به‌صورت مستقل توسعه، آزمایش و نگهداری شود. این معماری شامل چهار لایه‌ی اصلی است:

1. لایه دریافت و پردازش داده‌ها

در این بخش، کد منبع مخزن انتخاب‌شده و داده‌های متنی مرتبط (Pull Requests و Issues) از GitHub توسط API‌هایی که در اختیار می‌گذارند، استخراج می‌شوند. سپس پس از پردازش داده‌ها، آن‌ها شامل خلاصه‌های متنی برای هر Issue و Discussion می‌گردند. علاوه‌براین، مکانیزم برچسب‌گذاری خودکار برای شناسایی بدهی‌های فنی و code smells در سطح فایل‌ها و Issues پیاده‌سازی می‌شود.

2. لایهٔ بازیابی و رتبه‌بندی

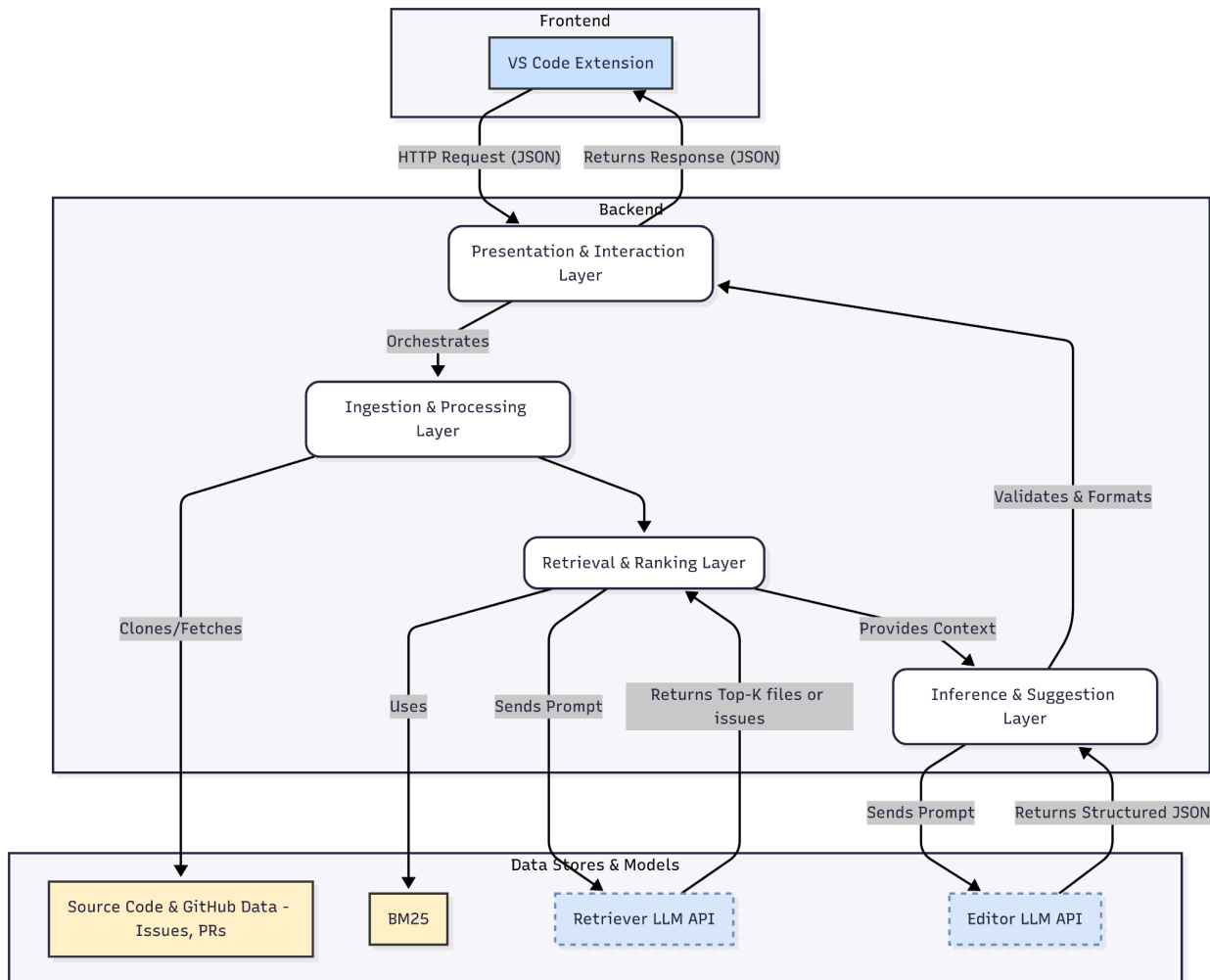
این لایه مسئول جست‌وجوی فایل‌ها و مسائل مشابه با استفاده از شاخص‌های BM25 **نیاز به منبع** و الگوریتم‌های رتبه‌بندی است. هدف آن ارائه‌ی مجموعه‌ای از اسناد و اطلاعات مرتبط (Context) برای مدل زبانی می‌باشد.

3. لایهٔ استنتاج و پیشنهاد

در این بخش، مدل زبانی بزرگ (LLM) از طریق چارچوب LangChain و با بهره‌گیری از پرامپت‌های طراحی‌شده، بر مبنای فایل هدف و context بازیابی‌شده، پیشنهادهایی برای اصلاح یا بهبود کد ارائه می‌دهد. خروجی این لایه به‌صورت JSON شامل کد تغییرات پیشنهادی، کد قدیمی که تغییر می‌کند و شماره خطوط است.

4. لایهٔ ارائه و تعامل

تعامل کاربر از طریق یک افزونه در محیط توسعه Visual Studio Code صورت می‌گیرد. این افزونه به‌عنوان Frontend عمل کرده و درخواست‌ها را به یک Backend API ارسال می‌کند. Backend وظیفه دارد جریان داده، جست‌وجوی زمینه و فراخوانی مدل زبانی را هماهنگ کرده و نتایج را به‌شکل استاندارد به افزونه بازگرداند. افزونه نیز تغییرات پیشنهادی را به‌صورت قابل‌مشاهده و تعاملی به کاربر نمایش می‌دهد.



۹-۳- معماری کلاینت/سرور و پروتکل ارتباطی Frontend-Backend

معماری سامانه به صورت کلاینت/سرور طراحی شده است تا وظایف سنگین پردازشی (از جمله جست و جوی اسناد و استنتاج مدل زبانی بزرگ) از محیط توسعه کاربر جدا شود. در این معماری، Frontend به صورت افزونه VS Code عمل می کند و رابط کاربر را فراهم می سازد، در حالی که Backend یک سرویس مبتنی بر FastAPI است که مسئولیت پردازش داده ها، اجرای مراحل بازیابی و فراخوانی مدل زبانی را بر عهده دارد.

پروتکل ارتباطی میان این دو بخش مبتنی بر HTTP/HTTPS و در قالب RESTful API پیاده سازی شده است. پیام ها به صورت JSON تبادل می شوند، زیرا این قالب سبک، قابل اعتبارسنجی و سازگار با ابزارهای متعدد است. درخواست ها از سمت کلاینت شامل اطلاعاتی نظیر مسیر فایل انتخابی، متن کامل فایل یا شماره Issue است و پاسخ ها شامل خروجی های ساخت یافته مدل (مانند JSON انتخاب فایل های مرتبط یا JSON پیشنهاد تغییرات).

پارامترهای ورودی	شرح عملکرد	Method	Endpoint
issue_text (متن مشکل)، files (فایل‌های مرتبط)	دریافت توضیح باگ و تولید patch پیشنهادی	POST	generate/
issue_id یا issue_text	استخراج فایل‌های مرتبط از متن issue	GET	files/
patch (پچ پیشنهادی)، file (مسیر فایل)	بررسی صحت ساختاری و نحوی patch	POST	validate/
بدون پارامتر	بررسی سلامت سرور و آماده‌به‌کار بودن سیستم	GET	health/

۳-۱۰- افزونه فرانت‌اند VS Code

افزونه VS Code به‌عنوان رابط تعاملی کاربر طراحی شده است. این افزونه دو وظیفه اصلی دارد:

1. **گردآوری داده ورودی:** کاربر می‌تواند یک فایل خاص یا یک Issue را برای بررسی انتخاب کند. افزونه محتوای فایل یا شناسه Issue را دریافت کرده و به‌صورت JSON به Backend ارسال می‌کند. همچنین کاربر می‌تواند بررسی خودکار (Auto-Check) یا بررسی دستی را فعال کند.
2. **نمایش خروجی‌ها:** نتایج برگشتی از Backend، شامل انتخاب فایل‌ها و مسائل مرتبط و پیشنهادها، در محیط ویرایشگر به کاربر نمایش داده می‌شوند. برای مثال، تغییرات پیشنهادی می‌توانند در قالب *Code Lens* یا *Diff View* نمایش یابند، به‌گونه‌ای که

توسعه‌دهنده بتواند تغییرات را بپذیرد یا رد کند.

پیاده‌سازی افزونه با استفاده از VS Code Extension API انجام می‌شود و قابلیت سفارشی‌سازی تجربه کاربر را از طریق رابط‌های گرافیکی و کلیدهای میان‌بر فراهم می‌سازد. افزونه هیچ پردازش سنگینی انجام نمی‌دهد و تنها مسئول هماهنگی میان کاربر و Backend است.

۳-۱۱- API سمت سرور برای تولید پاسخ‌های مدل زبانی بزرگ

بخش بک‌اند این سامانه، که با استفاده از چارچوب وب FastAPI در زبان پایتون توسعه یافته است، مسئولیت مدیریت کامل چرخه یک درخواست، از لحظه دریافت آن از افزونه VS Code تا آماده‌سازی و ارسال پاسخ نهایی را بر عهده دارد. این فرآیند در قالب یک پایپ‌لاین پردازشی دقیق و مرحله‌بندی‌شده اجرا می‌شود که در ادامه تشریح می‌گردد.

فرآیند با دریافت یک درخواست از کلاینت آغاز می‌شود که حاوی اطلاعاتی نظیر فایل هدف یا شناسه مشکل (Issue) است. پس از دریافت، سرویس بلافاصله وارد فاز پیش‌پردازش می‌شود. در این مرحله، داده‌های خام آماده‌سازی می‌شوند؛ این شامل استخراج مستندات فایل‌ها (FileDocs) و فراخوانی خلاصه‌ساز (Summarizer) برای متراکم‌سازی اطلاعات موجود در Issues است تا یک کانتکست (context) غنی و کارآمد شکل گیرد.

خلاصه‌ساز مسائل و بحث‌ها (Summarizer)

یکی از چالش‌های اصلی در استفاده از داده‌های استخراج‌شده از GitHub، طولانی بودن متن‌ها در Issues و Discussions است. بسیاری از این متن‌ها شامل توضیحات تکراری، بحث‌های فرعی یا اطلاعاتی هستند که برای مدل زبانی در مرحله استنتاج ضروری نیستند. به همین دلیل، یک ماژول خلاصه‌ساز طراحی و پیاده‌سازی شده است تا این داده‌های خام را به شکل فشرده و قابل استفاده در پرامپت‌ها تبدیل کند.

وظایف اصلی

1. Issue Summary:

بدنه اصلی هر Issue به یک خلاصه کوتاه و شفاف تبدیل می‌شود که ماهیت مشکل یا درخواست تغییر را بیان می‌کند.

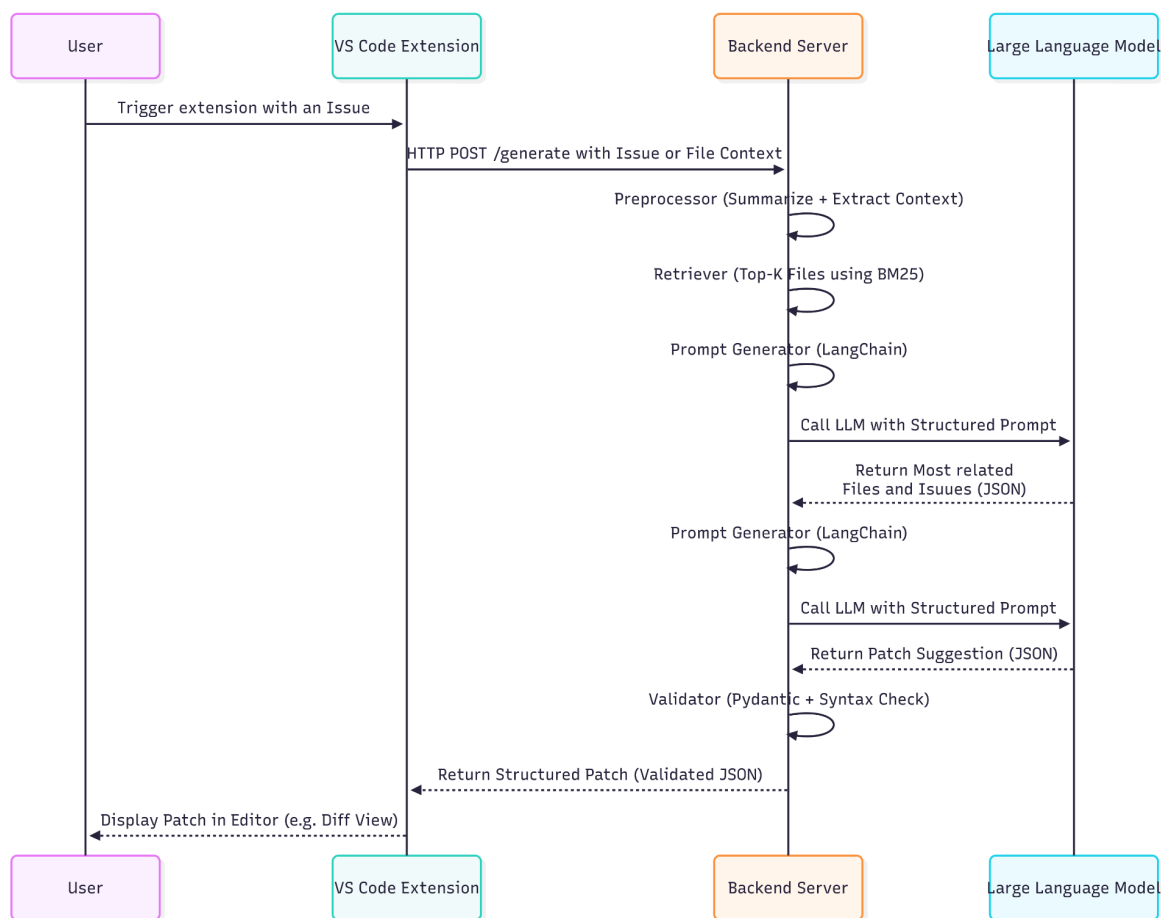
2. Discussion Summary:

نظرات و گفتگوهای کاربران پیرامون هر Issue یا Pull Request به یک نتیجه فشرده تقلیل می‌یابد. این خلاصه معمولاً شامل نتیجه‌گیری است.

روش پیاده‌سازی

- برای پیاده‌سازی Summarizer از مدل‌های زبانی بزرگ (LLM) در قالب LangChain Chains استفاده می‌شود.
 - ورودی Summarizer شامل متن کامل Issue یا مجموعه نظرات است.
 - مدل با استفاده از یک پرامپت خلاصه‌سازی که به صورت دقیق طراحی شده (مثلاً با تأکید بر استخراج مشکل اصلی و راه‌حل پیشنهادی) خروجی را در قالب یک متن کوتاه تولید می‌کند.
 - برای کنترل کیفیت، طول خروجی محدود (مثلاً ۲ تا ۳ جمله) نگه داشته می‌شود تا هنگام استفاده در پرامپت نهایی، از پرشدن بیهوده ظرفیت مدل جلوگیری گردد.
- با آماده شدن داده‌ها، مرحله بازیابی اطلاعات با استفاده از الگوریتم BM25 اجرا می‌شود تا مرتبط‌ترین اسناد از میان انبوه فایل‌ها و مشکلات شناسایی شوند. سپس، این اطلاعات منتخب در قالب یک پرامپت مهندسی‌شده و با زمینه محدود (bounded-context prompt) برای ارسال به مدل زبانی بزرگ آماده می‌گردد.
- گام بعدی، فراخوانی مدل زبانی (LLM Inference) است. پرامپت ساخته‌شده به یک مدل زبانی پیشرفته (مانند خانواده مدل‌های Gemini) ارسال شده و از آن خواسته می‌شود تا خروجی خود را در قالب یک ساختار JSON دقیق و از پیش تعریف‌شده ارائه دهد. پس از دریافت پاسخ از مدل، یک فرآیند اعتبارسنجی چندمرحله‌ای آغاز می‌شود. ابتدا، ساختار JSON دریافتی با اسکیمای (Schema) مورد انتظار مقایسه و صحت‌سنجی می‌شود. در صورت تأیید، تغییرات پیشنهادی به صورت مجازی (in-memory) بر روی کد اعمال شده و یک تحلیل نحوی (syntactic analysis) روی نتیجه انجام می‌شود تا از صحت ساختاری کد اطمینان حاصل شود. در این پروژه این قابلیت فقط برای زبان پایتون پیاده‌سازی شده است.

در نهایت، پس از عبور موفقیت‌آمیز از تمامی مراحل، نتایج نهایی—خواه فهرستی از فایل‌ها و مشکلات مرتبط باشد یا پیشنهادهای مشخص برای تغییر کد—بسته‌بندی شده و به عنوان پاسخ نهایی به افزونه VS Code بازگردانده می‌شود تا به کاربر نمایش داده شود.



۳-۳- مدل داده و منابع اطلاعاتی

برای طراحی و پیاده‌سازی سامانه، نیاز به تعریف دقیق منابع اطلاعاتی و داده‌هایی است که در مراحل مختلف مورد استفاده قرار می‌گیرند. این مدل داده، امکان یکپارچگی میان فایل‌های کد، Issues، و نتایج پردازش را فراهم می‌سازد.

نوع داده	شرح	فیلدهای کلیدی
Source File	فایل‌های کد منبع پروژه که ممکن است حاوی باگ باشند	path, content, language
Issue	توضیح و گزارش مشکل ثبت‌شده توسط کاربر یا توسعه‌دهنده	title, description, issue_id
Comments	گفتگوهای مرتبط با issue در GitHub شامل پیشنهادات، توضیحات و بازخوردها	comments[], author, timestamps
Summary	خلاصه‌ای متنی از اطلاعات موجود در issue و بحث‌ها	text, source_issue_id
FileDoc	مستندات یا توصیف‌های افزوده‌شده برای هر فایل (از مدل زبانی یا تحلیل)	file_path, summary, []functions

• فایل‌های کد (Source Files)

- هر فایل در مخزن کد به‌عنوان یک منبع اصلی در نظر گرفته می‌شود.
- **File Documentation (مستندات فایل‌ها):**

هر فایل شامل اطلاعات ساختاری زیر است:

- مسیر فایل (file path)
- توضیحات ماژول (module docstring)
- نام کلاس‌ها همراه با docstring و متدها
- توابع به همراه نام، پنج خط اول و پنج خط آخر بدنه تابع

این مستندات به صورت خلاصه‌سازی شده برای کاهش حجم کانتکست (context) در ارسال به مدل‌های زبانی استفاده می‌شوند، ولی همچنان اطلاعات کلیدی برای بازیابی فایل‌ها را حفظ می‌کنند.

نیاز به منبع

• مسائل و بحث‌ها (Issues & Discussions)

- هر Issue شامل شناسه یکتا (Issue Number)، عنوان، متن اصلی، و برچسب‌های مرتبط است. همچنین در پیاده‌سازی تفاوتی بین pull request و issue ها وجود ندارد. لذا از این به بعد هر جا به issue اشاره شد، فرض این است که می‌تواند یک pull request باشد.
- محتوای متنی کامنت‌ها و بحث‌های پیرامون هر Issue استخراج و در قالب یک Discussion Summary خلاصه می‌گردد.
- برای هر Issue یک Issue Summary نیز تولید می‌شود تا دید کلی نسبت به مشکل یا درخواست تغییر ارائه گردد.

• شاخص‌ها و ساختارهای بازیابی (Retrieval Indexes)

- هنگامی که کاربر تحلیلی را برای یک فایل هدف مشخص آغاز می‌کند، محتوای کامل آن فایل به عنوان Query برای شاخص BM25 استفاده می‌شود. این الگوریتم با استفاده از فرمول رتبه‌بندی خود، امتیازی برای هر سند موجود در شاخص (سایر فایل‌های کد و خلاصه‌های Issues) محاسبه می‌کند.
- این مرحله به عنوان یک بازیابی سطح بالا یا کلی عمل می‌کند. به این معنا که جستجو بر اساس تطابق کلمات کلیدی و شباهت آماری انجام می‌شود و هنوز درک عمیق معنایی (Semantic) از محتوا وجود ندارد. در واقع، این فاز مانند یک فیلتر سریع و کارآمد عمل می‌کند که از میان هزاران فایل و مشکل احتمالی، مجموعه‌ای کوچک و بسیار مرتبط‌تر از نامزدها را برای مراحل بعدی پایپ‌لاین آماده می‌سازد. این نامزدها سپس در اختیار مدل زبانی بزرگ قرار می‌گیرند تا تحلیل دقیق‌تر و ظریف‌تری (Fine-grained) روی آن‌ها انجام شود.
- خروجی این فرآیند، یک لیست رتبه‌بندی‌شده از K سند برتر است که بالاترین امتیاز شباهت لغوی را با فایل هدف داشته‌اند. این لیست شامل مسیر فایل‌ها و شناسه‌های مشکلاتی است که به عنوان مرتبط‌ترین نامزدها شناسایی شده‌اند.

• مقایسه BM25 و رویکرد ترکیبی (Hybrid Search)

- جستجوی لغوی (BM25): برای یافتن اسنادی که دقیقاً از همان کلمات کلیدی (نام متغیرها، توابع و...) استفاده می‌کنند، بسیار سریع و مؤثر است.
- جستجوی معنایی (Vector Search): برای یافتن اسناد مفهوماً مرتبط، حتی اگر از کلمات متفاوتی استفاده کرده باشند، قدرت بالایی دارد.

نقطه ضعف جستجوی معنایی این است که گاهی ممکن است اسنادی را بازایی کند که از نظر موضوعی کلی مرتبط هستند اما جزئیات فنی لازم را ندارند. به همین دلیل، پیشرفته‌ترین سیستم‌های جستجو امروزه از جستجوی ترکیبی (Hybrid Search) استفاده می‌کنند. در این رویکرد، نتایج هر دو روش BM25 و جستجوی شباهت برداری با یکدیگر ترکیب می‌شوند تا یک لیست نهایی رتبه‌بندی‌شده به دست آید که از نقاط قوت هر دو روش بهره می‌برد.

در پروژه حاضر، به دلیل سادگی پیاده‌سازی، سرعت بالا و عملکرد قابل قبول به عنوان یک مبنای اولیه، از BM25 استفاده شده است. با این حال، ارتقاء سیستم به یک مدل جستجوی ترکیبی، می‌تواند یکی از مسیرهای اصلی برای بهبود دقت و کارایی آن در کارهای آینده محسوب شود.

لایه دریافت و پردازش داده‌ها

این لایه نقطه آغازین جریان داده در سامانه محسوب می‌شود و وظیفه اصلی آن، جمع‌آوری، پالایش و آماده‌سازی داده‌ها برای مراحل بعدی است. منابع داده شامل دو بخش کلیدی‌اند: کد منبع مخزن و داده‌های متنی مرتبط با مدیریت توسعه نرم‌افزار (Pull Requests و Issues).

استخراج کد منبع

ابتدا مخزن انتخاب‌شده از GitHub کلون شده و کلیه فایل‌های موجود در آن در قالب ساختار درختی در دسترس قرار می‌گیرند. در این مرحله، فراداده‌هایی همچون مسیر فایل، زبان برنامه‌نویسی، تعداد خطوط و اندازه فایل نیز ذخیره می‌شوند. این اطلاعات علاوه بر محتوای خام، امکان انجام تحلیل‌های بعدی نظیر شناسایی نشانه‌های ضعف کد را فراهم می‌کنند.

گردآوری داده‌های متنی

در گام بعد، داده‌های متنی شامل Pull Requests، Issues و بحث‌های (Discussions) مربوط به هرکدام با استفاده از GitHub API استخراج می‌شوند. هر Pull Request و Issue همراه با محتوای

اصلی، نظرات و وضعیت فعلی آن‌ها ثبت می‌گردد. این داده‌ها به‌عنوان بخشی از زمینه معنایی برای مدل زبانی مورد استفاده قرار خواهند گرفت.

پردازش و خلاصه‌سازی

به‌منظور کاهش حجم داده و تمرکز بر اطلاعات کلیدی، هر Issue و Discussion به‌کمک مدل زبانی بزرگ یا الگوریتم‌های خلاصه‌سازی متنی به یک یا چند خلاصه فشرده تبدیل می‌شوند. این خلاصه‌ها در مراحل بعدی برای ایجاد پرامپت‌های مختصر و در عین حال معنادار مورد استفاده قرار می‌گیرند. بدین ترتیب، مدل به جای مواجهه با متن‌های طولانی و گاه غیرضروری، تنها به داده‌های اساسی دسترسی دارد.

لایه دریافت و پردازش داده‌ها، مواد خام موردنیاز سامانه را به داده‌هایی فشرده و قابل استفاده در مراحل بازایی و استنتاج تبدیل می‌کند. این لایه تضمین می‌کند که داده‌ها هم از نظر کمی و هم از نظر کیفی برای استفاده در مدل زبانی مناسب باشند.

۳-۵- راهبرد بازایی و رتبه‌بندی

راهبرد بازایی و رتبه‌بندی در سامانه‌ی پیشنهادی بر این اصل استوار است که صرفاً شباهت‌های سطحی یا متنی میان فایل‌ها و مسائل (Issues) برای یافتن ارتباط واقعی کافی نیست. بنابراین، فرآیند در دو گام متوالی طراحی شده است: **بازایی اولیه (Coarse Retrieval)** و **رتبه‌بندی دقیق‌تر (Fine Ranking)**.

در گام نخست، فایل یا Issue هدف به‌طور کامل به شاخص BM25 داده می‌شود تا مجموعه‌ای از نزدیک‌ترین فایل‌ها و مسائل مشابه بر اساس شباهت واژگانی استخراج گردد. این مرحله با وجود سرعت بالا، دقت لازم برای شناسایی مهم‌ترین زمینه‌ها را ندارد، زیرا شباهت متنی الزاماً به معنای ارتباط مفهومی یا رفع مشکل مشابه نیست. ازاین‌رو گام دوم یعنی رتبه‌بندی دقیق‌تر اهمیت پیدا می‌کند.

۳-۶- طراحی پرامپت و خروجی ساختاریافته

در رتبه‌بندی دقیق‌تر، سیستم وارد مرحله‌ای می‌شود که با ساخت یک پرسش دارای زمینه‌ی محدود محتوای انتخاب‌شده را به مدل زبانی منتقل می‌کند. این پرامپت شامل چند بخش اساسی است:

- دستورالعمل (Task Description): مشخص کردن هدف اصلی مانند «یافتن فایل‌ها یا مسائل مرتبط برای رفع باگ در فایل X».
- محتوای فایل یا issue درخواست شده برای پیشنهاد دهی
- نمای فشرده فایل‌ها (FileDocs): برای فایل‌های بازایی شده در مرحله قبل، نمای فشرده‌ای شامل خطوط ابتدایی و انتهایی، امضاهای توابع و کلاس‌ها، یا بخش‌های کلیدی دیگر در پرامپت گنجانده می‌شود که پیشتر با دقت بیشتری توضیح داده شد.
- ورودی مشکلات ثبت شده (Issue Entries): برای Issues، خلاصه‌ی بدنه (Issue Summary) به همراه خلاصه‌ی بحث‌ها (Discussion Summary) قرار داده می‌شود تا مدل زبانی بتواند ارزیابی کند که این مسئله تا چه حد با فایل هدف مرتبط است.

پس از ترکیب این داده‌ها در یک پرسش با استفاده از کتابخانه Langchain مدل زبانی بزرگ (LLM) فراخوانی می‌شود. ویژگی مهم این مرحله، استفاده از خروجی ساختاریافته (Structured Output) است؛ بدین معنا که مدل ملزم می‌گردد نتیجه را نه به صورت متن آزاد بلکه در قالب JSON استاندارد بازگرداند. این قابلیت توسط ابزار Langchain مهیا می‌شود. این JSON حاوی لیستی از فایل‌ها و مسائل پیشنهادی است که بر اساس قضاوت مدل، بیشترین ارتباط را با فایل هدف دارند.

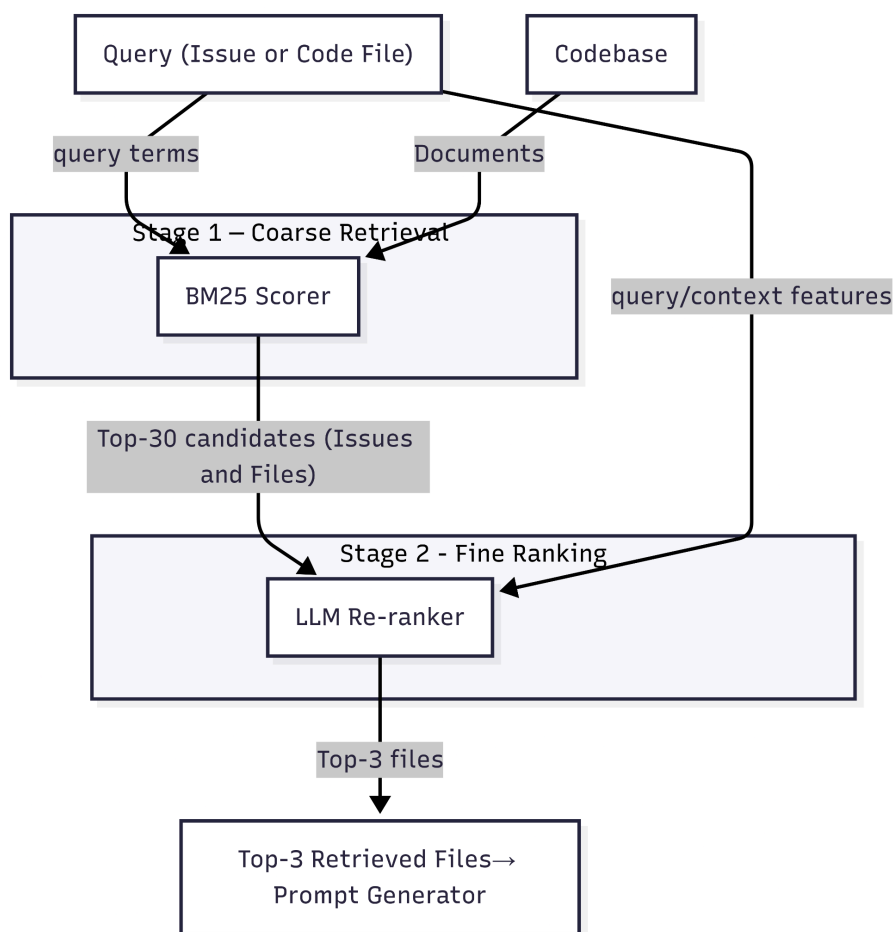
این طراحی چند مزیت کلیدی دارد:

- بهینه‌سازی ظرفیت مدل: از آن‌جا که مدل زبانی ظرفیت محدودی در ورودی دارد، ارائه‌ی داده‌ها در قالب خلاصه‌شده (Summaries و FileDocs) سبب می‌شود اطلاعات غیرضروری حذف شوند.
- افزایش دقت در انتخاب: ترکیب داده‌های چندمنبعی (محتوای فایل، خلاصه‌ی Issues، بحث‌های پیرامون) به مدل امکان می‌دهد انتخاب‌های خود را بر مبنای ارتباط مفهومی انجام دهد، نه صرفاً شباهت سطحی.
- قابلیت پردازش ماشینی: بازگشت خروجی در قالب JSON موجب می‌شود که سیستم بتواند بدون دخالت انسانی انتخاب‌ها را پردازش و در مراحل بعدی (مانند تولید پیشنهاد اصلاح کد) به کار گیرد.

روش پیاده‌سازی:

پرامپت‌ها با استفاده از LangChain PromptTemplate طراحی شدند. داده‌های موردنیاز (FileDocs، Issue Summaries و Discussion Summaries) در قالب یک پرامپت مهندسی‌شده به مدل داده شدند. برای دریافت خروجی ساختاریافته، از قابلیت with_structured_output() در LangChain استفاده شد که خروجی را با شمای Pydantic هماهنگ می‌کند. این کار اطمینان می‌دهد خروجی مدل به‌صورت JSON بازگردد.

در مجموع، مرحله رتبه‌بندی دقیق‌تر، با بهره‌گیری از پرسش خاص و خروجی استاندارد، حلقه‌ی واسطی میان جست‌وجوی اولیه و استنتاج نهایی است و سعی می‌کند که مرتبط‌ترین و ارزشمندترین فایل‌ها و مسائل را با مدل زبانی شناسایی کند.



۳-۷- تولید پیشنهاد و اعمال تغییرات

مرحله تولید پیشنهاد و اعمال تغییرات در سامانه بر مبنای طراحی یک ماژول ویرایش کد انجام می‌شود که هدف آن ایجاد بخش‌هایی از کد است که به صورت دقیق و ساختاریافته برای فایل‌های شناسایی شده، تولید شده است. این بخش از پژوهش، با الهام از روش‌های پیشنهادی در **SWE-Fixer**، به گونه‌ای طراحی شده که هم قابلیت پیاده‌سازی در یک سامانه‌ی عملیاتی را داشته باشد و هم از منظر دقت و کارایی قابل دفاع باشد. **نیاز به منبع**

ورودی شامل عناصر زیر است:

- فایل ورودی یا issue با محتوای کامل.
- مجموعه‌ای از فایل‌های مرتبط که در مرحله بازبازی انتخاب شده‌اند.
- خلاصه مسائل (Issue Summary) و بحث‌ها (Discussion Summary) که به عنوان زمینه تکمیلی عمل می‌کنند.
- دستورالعمل (Task Instruction) که بیان می‌کند مدل باید چه تغییری در کد ایجاد کند.

مدل زبانی با استفاده از داده‌های ورودی، تغییرات لازم را در قالب قسمت‌هایی از کد پیشنهاد می‌دهد. برخلاف بازنویسی کامل فایل، تغییرات به صورت بخش‌های اصلاحی کوچکی تعریف می‌شوند. هر بخش شامل سه جز اصلی است:

1. **مسیر فایل (File Path):** مشخص می‌کند تغییر در کدام فایل باید اعمال شود.
2. **بخش کد اصلی (Code Snippet to be Modified):** قطعه کدی از فایل (به همراه شماره خطوط) که باید تغییر کند.
3. **بخش کد اصلاح شده (Edited Code Snippet):** قطعه کدی که جایگزین بخش اصلی می‌شود.

مطالعاتی مانند SWE-Fixer نشان داده‌اند که ارائه بخشی از کد در قالب ساختاریافته عملکرد مدل را نسبت به خروجی‌های متنی آزاد به طور چشمگیری بهبود می‌دهد. **نیاز به منبع**

نمونه خروجی JSON:

یک خروجی نمونه از این مرحله می‌تواند به صورت زیر باشد:

```
{
  "edited_code": [
```

```

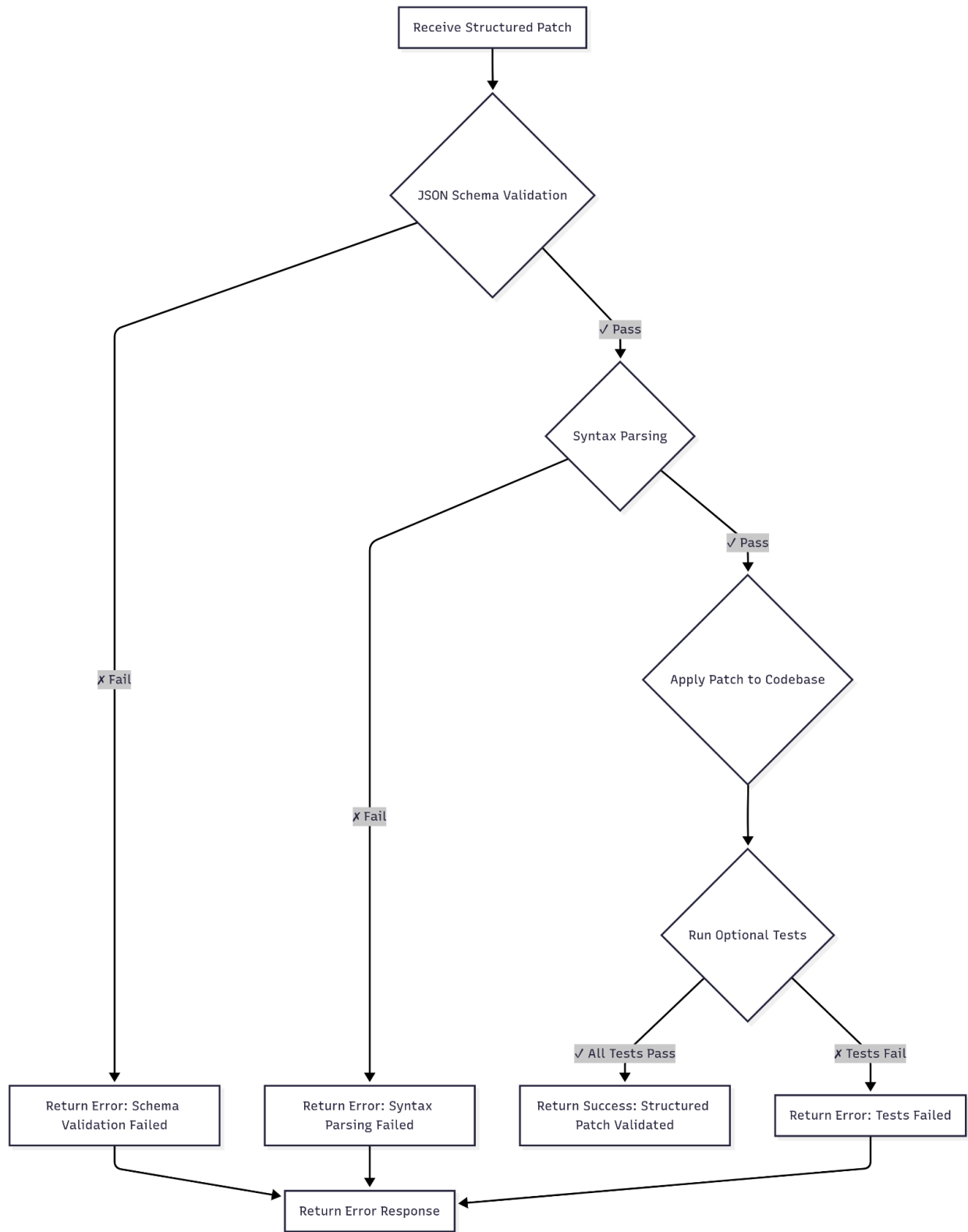
{
    "file": "src/validator.py",
    "code_snippet_to_be_modified": "
        102     result = process_input(data)
        103     return result",
    "edited_code_snippet": "
        if data is None:
            raise ValueError('Input data cannot be None')
        result = process_input(data)
        return result"
}
]
}

```

با این طراحی، مرحله ویرایش کد قادر است تصحیح‌های کوچک، دقیق و قابل استفاده در محیط‌های واقعی توسعه نرم‌افزار ارائه دهد. این انتخاب باعث می‌شود فرایند اصلاح هم با الزامات فنی پروژه‌های واقعی سازگار باشد و هم قابلیت ارزیابی دقیق در آزمایش‌های پژوهشی (مانند SWE-Bench) را داشته باشد.

۳-۸- اعتبارسنجی و کنترل کیفیت

پس از تولید بخش‌های پیشنهادی توسط مدل زبانی، مرحله اعتبارسنجی و کنترل کیفیت به‌عنوان فیلتری ضروری عمل می‌کند تا اطمینان حاصل شود تغییرات پیشنهادی نه‌تنها از نظر نحوی معتبر هستند بلکه از منظر یکپارچگی و کارکرد نیز قابل قبول‌اند. این مرحله در سه سطح پیاده‌سازی شده است: اعتبارسنجی ساختاری، اعتبارسنجی نحوی، و کنترل کیفیت محتوایی.



اعتبارسنجی ساختاری:

در گام نخست، خروجی مدل که در قالب JSON ساخت‌یافته ارائه می‌شود، در برابر یک شمای مشخص بررسی می‌گردد. این کار موجب می‌شود که تنها تغییراتی پذیرفته شوند که دقیقاً مطابق با قالب تعریف‌شده (مانند داشتن کلیدهای `file`، `code_snippet_to_be_modified` و `edited_code_snippet`) باشند. این بررسی از بروز خطاهای ناشی از تولید خروجی‌های ناقص یا ناسازگار جلوگیری می‌کند.

اعتبارسنجی نحوی:

در گام دوم، تغییرات پیشنهادی به‌طور موقت بر روی نسخهٔ محلی فایل‌ها اعمال می‌شوند و سپس یک تحلیل نحوی (Syntax Analysis) بر مبنای زبان برنامه‌نویسی مربوطه صورت می‌گیرد. برای زبان پایتون، این بررسی با استفاده از کتابخانه `ast` انجام می‌شود که در صورت وجود خطاهای نحوی (مانند تورفتگی نادرست یا دستورات ناقص) آن‌ها را گزارش می‌دهد. این مرحله تضمین می‌کند که حتی اگر بخش پیشنهادی از نظر ساختار JSON معتبر باشد، در سطح کد قابل اجرا باقی بماند.

کنترل کیفیت محتوایی:

- در گام سوم، تغییرات از منظر کیفیت بررسی می‌شوند. این بررسی می‌تواند شامل موارد زیر باشد:
- اطمینان از آن‌که خطوط تغییر داده‌شده با خطوط اصلی هم‌خوانی دارند و هیچ بخش نامرتبیتی حذف یا جایگزین نشده است.
 - شناسایی هم‌پوشانی یا تضاد میان بخش‌های مختلف؛ برای مثال، اگر دو بخش پیشنهادی قصد تغییر یک بخش یکسان از کد را داشته باشند.
 - ارزیابی سطحی عملکرد کد پس از تغییر، از طریق اجرای تست‌های واحد موجود در مخزن (در صورت در دسترس بودن).

جمع‌بندی

در این فصل، سامانهٔ پیشنهادی برای پیشنهاد و اعمال خودکار اصلاحات در کد به‌صورت گام‌به‌گام تشریح شد. ابتدا معماری کلی سیستم و ساختار چندلایهٔ آن معرفی گردید. سپس، نقش هر مؤلفه از جمله افزونهٔ VS Code به‌عنوان رابط کاربر و سرویس FastAPI در سمت سرور توضیح داده شد.

همچنین ابزارهای کلیدی پیاده‌سازی شامل FastAPI برای ساخت سرویس‌های وب، Pydantic برای اعتبارسنجی داده‌ها، LangChain برای مدیریت پرامپت‌ها و خروجی‌های ساخت‌یافته، و API‌های GitHub برای گردآوری داده‌های خام معرفی شدند. افزونهٔ VS Code به‌عنوان بخش فرانت‌اند، تعامل ساده و بی‌واسطهٔ کاربر با بک‌اند را امکان‌پذیر ساخته است.

در ادامه، ماژول Summarizer توضیح داده شد که وظیفهٔ خلاصه‌سازی Issues و Discussions را بر عهده دارد و به‌عنوان ابزار کاهش حجم داده و بهینه‌سازی ورودی مدل زبانی عمل می‌کند. سپس راهبرد بازیابی اطلاعات با استفاده از الگوریتم BM25 شرح داده شد که به‌عنوان یک فیلتر سریع و کارآمد، نزدیک‌ترین فایل‌ها و مسائل را برای تحلیل دقیق‌تر انتخاب می‌کند. بر پایهٔ این نتایج، یک پرامپت مهندسی‌شده و زمینهٔ محدود به مدل زبانی ارسال می‌شود و خروجی در قالب JSON بازگردانده می‌شود.

در مرحلهٔ بعد، بخش تولید پیشنهاد و اعمال تغییرات توضیح داده شد که با الهام از رویکردهای موجود مانند *SWE-Fixer*، تغییرات را به‌صورت بخش‌های کوچک و هدفمند تولید می‌کند تا هم قابل پردازش باشد و هم در محیط توسعه به‌سادگی اعمال شود.

به‌طور کلی، این فصل نشان داد که چگونه اجزای مختلف پروژه از مرحلهٔ استخراج داده‌های خام تا تولید و اعتبارسنجی پیشنهادهای اصلاحی، در قالب یک پایپ‌لاین یکپارچه با یکدیگر تعامل می‌کنند.

فصل چهارم: نتایج

۴-۱ جمع‌بندی

در این پژوهش مسئله‌ی بدهی فنی و چالش‌های ناشی از بازبینی دستی کد به عنوان یکی از موانع مهم در توسعه‌ی نرم‌افزارهای بزرگ مورد بررسی قرار گرفت. هدف اصلی، طراحی و پیاده‌سازی سامانه‌ای بود که بتواند فرآیند تشخیص و اصلاح کد را به صورت خودکار و هوشمند انجام دهد و در عین حال با محیط توسعه‌ی یکپارچه‌ی برنامه‌نویسان ادغام شود.

روش پیشنهادی بر پایه‌ی استفاده از مدل‌های زبانی بزرگ (LLMs) همراه با تکنیک‌های بازیابی اطلاعات و معماری سرویس‌گیرنده/سرویس‌دهنده بنا نهاده شد. اجزای کلیدی سامانه شامل ماژول خلاصه‌ساز (Summarizer) برای کاهش حجم داده‌های متنی، الگوریتم BM25 برای رتبه‌بندی و بازیابی اسناد، چارچوب LangChain برای طراحی پرامپت و دریافت خروجی ساختاریافته، و همچنین Pydantic و Syntax Analysis برای اعتبارسنجی نتایج بود.

با پیاده‌سازی این معماری، سامانه توانست به صورت مرحله‌ای فرآیندهای مختلف شامل دریافت و پردازش داده، خلاصه‌سازی متون، بازیابی و رتبه‌بندی مسائل مرتبط، تولید پیشنهادهای اصلاحی، و در نهایت اعتبارسنجی و کنترل کیفیت تغییرات را انجام دهد. بدین ترتیب، توسعه‌دهندگان قادر شدند بدون صرف زمان طولانی برای بازبینی دستی، پیشنهادهای اصلاحی دقیق و ساختاریافته دریافت کنند.

نتایج حاصل نشان داد که این سامانه می‌تواند کاهش قابل توجهی در خطاهای انسانی ایجاد کرده و بهبود محسوسی در کیفیت و یکپارچگی کد فراهم آورد. همچنین یکپارچه‌سازی مستقیم با محیط VS Code موجب شد ابزار پیشنهادی علاوه بر جنبه‌ی پژوهشی، در عمل نیز برای توسعه‌دهندگان کاربردی و کارآمد باشد.

۴-۲ نتایج کلیدی

- کاهش زمان بازبینی کد و کمک به شناسایی سریع‌تر خطاها و مشکلات ساختاری.
- ترکیب روش‌های بازیابی اطلاعات با مدل‌های زبانی بزرگ که موجب بهبود دقت پیشنهادهای در مقایسه با رویکردهای صرفاً مبتنی بر LLM شد.
- فراهم‌سازی خروجی ساختاریافته (در قالب JSON) همراه با اعتبارسنجی نحوی و ساختاری برای اطمینان از کیفیت تغییرات پیشنهادی.

۳-۴ محدودیت‌ها

با وجود دستاوردهای این پژوهش، برخی محدودیت‌ها همچنان وجود دارد:

- وابستگی نتایج به کیفیت داده‌های استخراج‌شده از Issues و Discussions در GitHub.
- تمرکز اصلی سامانه بر زبان برنامه‌نویسی Python و عدم پوشش زبان‌های متنوع.
- نیاز به منابع محاسباتی قوی و هزینه‌ی بالا برای استفاده‌ی مستمر از مدل‌های زبانی بزرگ.
- عدم ارزیابی گسترده بر روی پروژه‌های صنعتی بزرگ که می‌تواند چالش‌هایی در زمینه‌ی مقیاس‌پذیری ایجاد کند.

فصل ۵ نتیجه‌گیری و پیشنهادها

۵-۱- جمع‌بندی

این پژوهش نشان داد که ترکیب مدل‌های زبانی بزرگ با تکنیک‌های بازیابی اطلاعات می‌تواند گامی مؤثر در جهت خودکارسازی فرآیند بازبینی و اصلاح کد باشد. رویکرد پیشنهادی علاوه بر بهبود کیفیت نرم‌افزار، توانست نقش موثری در کاهش هزینه‌ها، افزایش بهره‌وری، و بهبود تجربه‌ی توسعه‌دهندگان ایفا کند. با توجه به پیشرفت‌های سریع در حوزه‌ی LLMها، می‌توان انتظار داشت که در آینده‌ی نزدیک این ابزارها به بخشی جدایی‌ناپذیر از چرخه‌ی توسعه‌ی نرم‌افزار تبدیل شوند.

۵-۲- پیشنهادهایی برای ادامه پژوهش

در ادامه، مسیرهایی برای توسعه و تکمیل پژوهش پیشنهاد می‌شود:

گسترش دامنه‌ی زبانی و فنی

- پشتیبانی از زبان‌های برنامه‌نویسی دیگر مانند C، ++Java، و JavaScript.
- استفاده از تحلیل معنایی (Semantic Analysis) در کنار تحلیل نحوی جهت افزایش دقت تشخیص خطاها.

بهبود دقت و کارایی

- بهره‌گیری از Hybrid Search (ترکیب BM25 با جستجوی برداری مبتنی بر Embedding).
- Fine-tuning مدل‌های زبانی بر روی داده‌های تخصصی حوزه‌ی نرم‌افزار برای انطباق بیشتر با نیازهای برنامه‌نویسان.

ارزیابی جامع‌تر

- انجام آزمایش‌های گسترده بر روی مجموعه‌داده‌های استاندارد مانند SWE-Bench.
- مقایسه‌ی عملکرد سامانه با ابزارهای موجود همچون RepoBugs، SWE-Fixer، و InferFix.

بررسی ابعاد انسانی و صنعتی

- مطالعه‌ی تجربی برای سنجش اثر ابزار بر تجربه‌ی توسعه‌دهندگان.

- Blaauwendraad, R. (2023). *Evaluating large language models for code smell detection* (Master's thesis). Delft University of Technology
- Chen, Z., Wang, H., & Liu, Y. (2024). *When large language models confront repository-level automatic program repair: How well they done?* arXiv preprint .arXiv:2404.07864
- Jin, M., Shahriar, S., Tufano, M., Shi, X., Lu, S., Sundaresan, N., & Svyatkovskiy, A. (2023). *InferFix: End-to-end program repair with LLMs*. arXiv preprint .arXiv:2303.07263
- Kim, S., Kim, H., Kim, J., Kim, H., & Kim, T. (2023). *Improving patch optimization with buggy block for multi-hunk program repair*. In *Proceedings of the 45th International Conference on Software Engineering (ICSE 2023)* (pp. 1402–1414). .IEEE
- Li, F., Jiang, J., Sun, J., & Zhang, H. (2024). *Hybrid automated program repair by combining large language models and program analysis (GIANTREPAIR)*. arXiv preprint arXiv:2406.00992
- Mu, T., Yu, H., Yang, J., Zhang, X., Chen, Y., & Zhang, W. (2023). *EXPEREREPAIR: Dual-memory enhanced LLM-based repository-level program repair*. arXiv preprint arXiv:2311.05636
- Sheikhaei, S., Bavota, G., & Di Penta, M. (2023). *Understanding the effectiveness of LLMs in automated self-admitted technical debt identification*. In *Proceedings*

of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE 2023) (pp. 151–162). IEEE

- Shivashankar, V., & Martini, A. (2024). *TD-Suite: All batteries included framework for technical debt classification*. In *Proceedings of the 46th International Conference on Software Engineering (ICSE 2024)*. ACM
- Wadhwa, B., Sharma, S., & Goyal, N. (2023). *Frustrated with code quality issues? LLMs can help!* arXiv preprint arXiv:2308.12345
- Wang, Z., Huang, W., Chen, C., & Zhang, L. (2024). *Using developer discussions to guide fixing bugs in software*. In *Proceedings of the 46th International Conference on Software Engineering (ICSE 2024)*. ACM
- Xie, X., Sun, Y., Li, Q., & Zhang, H. (2023). *SWE-Fixer: Training open-source LLMs for effective and efficient GitHub issue resolution*. arXiv preprint .arXiv:2312.05194
- Xu, H., Wang, X., Gu, X., & Hu, Z. (2023). *ThinkRepair: Self-directed automated program repair*. arXiv preprint arXiv:2311.08344
- Yetistiren, E., Koc, M., & Arslan, A. (2023). *Evaluating the code quality of AI-assisted code generation: An empirical study on Copilot, ChatGPT, and Amazon CodeWhisperer*. arXiv preprint arXiv:2305.04047
- Zubair, M., Arora, R., Ahmad, R., & Nadi, S. (2025). *The use of large language models for program repair: Opportunities and challenges*. *Journal of Systems and Software*, 206, 111920. <https://doi.org/10.1016/j.jss.2024.111920>

